



GoDigital Payment Gateway

Payment API Integration Guide

This document defines the merchant-facing payment APIs for integrating with the Merchant Wallet Payment Gateway platform. It covers security, request and response standards, payment initiation endpoints, transaction status lookup, and asynchronous callbacks.

Field	Value
Document version	1.0
Audience	Merchant integration teams, backend engineers, solution architects, and support teams
Scope	Payment APIs only: C2B push collection, B2C disbursement, transaction status query, and callbacks
Channel model	Merchant requests specify the target providerCode only; integration routing is selected internally by the platform
Currency	TZS – Tanzania Shillings

Supported providerCode values

Provider code	Description
YAS	Requests and balances associated with the YAS wallet tree.
VODACOM	Requests and balances associated with the Vodacom wallet tree.
HALOTEL	Requests and balances associated with the Halotel wallet tree.
AIRTEL	Requests and balances associated with the Airtel wallet tree.

1. Overview

The Merchant Wallet Payment Gateway provides a single merchant-facing API surface for payment processing across multiple mobile money networks. Each merchant is onboarded with a wallet hierarchy made of one merchant parent wallet, provider total wallets, and provider leaf wallets for collection and disbursement.

- C2B collection increases the relevant provider collection wallet.
- B2C disbursement decreases the relevant provider disbursement wallet.
- The merchant parent wallet shows the total balance across all provider wallets.
- The provider total wallet shows the total balance across that provider's child wallets.
- For asynchronous providers, an acknowledgement does not by itself mean final success. Final wallet posting is done only after a final success outcome is recorded.

1.1 Base URL

Production: <https://{SERVER-IP}/api/v1>

UAT: <https://{SERVER-IP}/api/v1>

1.2 API style

Standard	Rule
Protocol	HTTPS only
Payload format	JSON UTF-8
Authentication	Bearer token plus HMAC request signature
Idempotency	Required for payment creation endpoints
Date and time format	ISO-8601 with timezone in responses
Amounts	Decimal numbers represented without thousands separators

1.3 Common response envelope

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "message": "Request processed successfully",
  "requestId": "a15c1e07-5e8f-4d45-a533-f1a57ad7c9d9",
  "timestamp": "2026-04-19T21:15:33+03:00",
  "data": { }
}
```

2. Security and request integrity

Merchant-facing APIs do not use certificate signing. Instead, each merchant integration uses OAuth2 client credentials, JWT bearer access tokens, HMAC request signing, timestamps, nonces, and idempotency keys.

2.1 Required headers for payment APIs

Header	Description
--------	-------------

Authorization	Bearer access token.
X-Client-Id	Client identifier issued during onboarding.
X-Request-Id	Unique request identifier. Must be unique per request.
X-Idempotency-Key	Required for payment initiation requests. Reusing the same key with the same payload returns the original result.
X-Timestamp	Request creation timestamp.
X-Nonce	Unique random nonce used for replay protection.
X-Content-SHA256	Base64 SHA-256 digest of the raw request body.
X-Signature	Base64 HMAC-SHA256 signature of the canonical request string.

2.2 Canonical request string

```
POST
/api/v1/payments/disbursements
{clientId}
{requestId}
{timestamp}
{nonce}
{contentSha256}
```

2.3 Idempotency behavior

For payment creation endpoints, the gateway compares the incoming idempotency key and request hash. If the exact same request is received again, the original transaction result is returned. If the same idempotency key is reused with a different payload, the request is rejected.

2.4 Token endpoint

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_id={clientId}&client_secret={clientSecret}
```

3. Payment data model

Field	Meaning
requestId	Merchant-generated unique request reference for API traceability.
transactionId	Gateway-generated immutable transaction identifier.
providerCode	Target provider or network wallet for posting, for example YAS or VODACOM.
reference	Merchant business reference such as invoice number, order number, or payout reference.
externalReference	Optional external or upstream reference where applicable.

amount	Transaction amount.
currency	Transaction currency, normally TZS.
msisdn	Customer or beneficiary mobile number in international format.
callbackUrl	Merchant endpoint for final asynchronous notification.

3.1 Transaction states

State	Meaning
RECEIVED	Request accepted by API boundary.
AUTHENTICATED	Security and client validation completed.
VALIDATED	Business rules and payload validation completed.
FUNDS_RESERVED	Applicable mainly to B2C; amount reserved from the provider disbursement wallet.
ACKNOWLEDGED	Provider or gateway accepted the request for processing.
PENDING_FINAL	Final provider outcome is still pending.
SUCCESS	Transaction completed successfully and ledger posting applied.
FAILED	Transaction failed; any reservation is released.
REVERSED	Successful reversal completed against the original transaction.
RECONCILING	The platform is reconciling because final provider outcome is delayed or inconsistent.

4. Payment APIs

The following APIs are in scope for this integration guide:

- B2C disbursement
- C2B push collection
- Transaction status query
- Callbacks for final status notification

4.1 Create B2C disbursement

Used by a merchant to send funds from its provider-specific disbursement wallet to a customer mobile wallet. The merchant identifies the target provider or network with providerCode. The gateway selects the internal route.

Endpoint: POST /api/v1/payments/disbursements

Request example

```
{
  "requestId": "6e3c1d3d-6b96-4e08-8a7d-c4968ea6905a",
  "providerCode": "YAS",
  "amount": 25000.00,
  "currency": "TZS",
```

```
"msisdn": "255713123456",
"reference": "PAYOUT-1001",
"callbackUrl": "https://merchant.example.com/api/payment-result",
"narration": "Customer refund"
}
```

Success response example

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "message": "Disbursement accepted for processing",
  "requestId": "6e3c1d3d-6b96-4e08-8a7d-c4968ea6905a",
  "timestamp": "2026-04-19T21:30:05+03:00",
  "data": {
    "transactionId": "TXN-20260419-000021",
    "providerCode": "YAS",
    "transactionStatus": "FUNDS_RESERVED",
    "reservedAmount": 25000.00,
    "currency": "TZS"
  }
}
```

Request field definitions

Field	Description
requestId	Required. Merchant-generated unique identifier.
providerCode	Required. Must be enabled for the merchant and mapped to a valid provider profile.
amount	Required. Must be greater than zero and within merchant limits.
currency	Required. Normally TZS.
msisdn	Required. Beneficiary mobile number in international format.
reference	Required. Merchant reference used for business traceability.
callbackUrl	Optional if already configured at merchant level; request value overrides only if allowed by policy.
narration	Optional merchant note or payment description.

For B2C, the platform first checks the merchant’s provider disbursement wallet. If sufficient funds are available, the amount is reserved. On final provider success, the reservation is consumed and ledger posting is completed. On failure, the reservation is released.

4.2 Create C2B push collection

Used by a merchant to initiate a push request to a customer. The final collection outcome is reported asynchronously through the merchant callback endpoint.

Endpoint: POST /api/v1/payments/collections/push

Request example

```
{
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "providerCode": "VODACOM",
  "amount": 10000.00,
  "currency": "TZS",
  "msisdn": "255754123456",
  "reference": "INV-1001",
  "callbackUrl": "https://merchant.example.com/api/payment-result",
  "narration": "Invoice INV-1001"
}
```

Acknowledgement response example

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "message": "Collection request accepted",
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "timestamp": "2026-04-19T21:34:41+03:00",
  "data": {
    "transactionId": "TXN-20260419-000022",
    "providerCode": "VODACOM",
    "transactionStatus": "ACKNOWLEDGED",
    "currency": "TZS"
  }
}
```

An acknowledgement only confirms that the push request was accepted for processing. The merchant must wait for the callback or query the transaction status endpoint to determine the final outcome. On final success, the platform credits the provider collection wallet and updates the parent total balances.

4.3 Query transaction status

Used by a merchant to retrieve the latest normalized status of a payment request.

Endpoint: GET /api/v1/payments/{transactionId}

Response example

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "message": "Transaction retrieved successfully",
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "timestamp": "2026-04-19T21:39:09+03:00",
  "data": {
    "transactionId": "TXN-20260419-000022",
    "providerCode": "VODACOM",
    "operation": "C2B_USSD_PUSH",
    "transactionStatus": "SUCCESS",
    "amount": 10000.00,
    "currency": "TZS",
    "msisdn": "255754123456",
  }
}
```

```
{
  "reference": "INV-1001",
  "providerTransactionId": "919994765",
  "providerReceiptNo": "5HN31LXJQAX",
  "finalizedAt": "2026-04-19T21:38:57+03:00"
}
```

5. Validation and error handling

Code	Meaning
PGW-1001	Invalid request payload.
PGW-1002	Unsupported or disabled providerCode for the merchant.
PGW-1003	Invalid MSISDN format.
PGW-1004	Duplicate requestId or idempotency conflict.
PGW-1005	Amount exceeds configured merchant limits.
PGW-1006	Insufficient available balance in the provider disbursement wallet.
PGW-1007	Authentication failed.
PGW-1008	Signature validation failed.
PGW-1009	Replay protection failed due to nonce or timestamp validation.
PGW-1010	Transaction not found.
PGW-1099	General processing error.

Example error response

```
{
  "status": "FAILED",
  "code": "PGW-1006",
  "message": "Insufficient available balance in disbursement wallet",
  "requestId": "6e3c1d3d-6b96-4e08-8a7d-c4968ea6905a",
  "timestamp": "2026-04-19T21:30:05+03:00",
  "data": null
}
```

6. Callback specification

Callbacks are mandatory for asynchronous finalization. The merchant must expose an HTTPS endpoint capable of accepting final transaction updates from the platform. The platform retries failed callback delivery attempts according to the configured retry policy.

6.1 Merchant callback endpoint requirements

Requirement	Description
Protocol	HTTPS only.
Method	POST
Content type	application/json
Response	Return HTTP 200 as soon as the callback is accepted for processing.
Idempotency	Merchant endpoint must safely handle duplicate callbacks for the same transactionId.
Timeout handling	The platform retries on timeout or non-2xx responses.

6.2 Callback payload

```
{
  "eventType": "PAYMENT_FINALIZED",
  "timestamp": "2026-04-19T21:38:57+03:00",
  "data": {
    "transactionId": "TXN-20260419-000022",
    "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
    "providerCode": "VODACOM",
    "operation": "C2B_USSD_PUSH",
    "transactionStatus": "SUCCESS",
    "amount": 10000.00,
    "currency": "TZS",
    "msisdn": "255754123456",
    "reference": "INV-1001",
    "providerTransactionId": "919994765",
    "providerReceiptNo": "5HN31LXJQAX",
    "message": "Transaction finalized successfully"
  }
}
```

6.3 Callback signature headers

Header	Description
X-Callback-Id	Unique callback delivery identifier.
X-Callback-Timestamp	Callback creation timestamp.
X-Callback-Content-SHA256	Base64 SHA-256 digest of the callback body.
X-Callback-Signature	Base64 HMAC-SHA256 signature using the merchant callback secret.

6.4 Merchant callback acknowledgement

```
HTTP/1.1 200 OK
Content-Type: application/json

{
```



```
"status": "RECEIVED",  
"message": "Callback accepted"  
}
```

6.5 Retry policy

If the merchant endpoint does not return HTTP 200 within the configured timeout, or if the response is non-2xx, the platform schedules retries. Repeated failures are visible in the callback logs available to administrators and merchants.

7. End-to-end processing summary

7.1 B2C disbursement

1. Merchant sends a signed disbursement request.
2. Gateway authenticates and validates the request.
3. Gateway checks the provider disbursement wallet and reserves funds.
4. Gateway submits the request to the selected internal route.
5. Gateway returns an acknowledgement response.
6. Provider sends final result or the platform resolves the result via status query.
7. On success, reservation is consumed and ledger posting is completed.
8. On failure, reservation is released.
9. Merchant callback is delivered.

7.2 C2B push collection

10. Merchant sends a signed collection push request.
11. Gateway validates and submits to the selected internal route.
12. Gateway returns an acknowledgement response.
13. Customer completes or rejects the request on the mobile channel.
14. Provider final result is received.
15. On success, the provider collection wallet is credited and total balances are updated.
16. Merchant callback is delivered.

8. Operational notes

Topic	Guidance
Timeouts	Merchants should use sensible client timeouts and rely on status query or callback for final results.
Duplicate handling	Merchants must treat transactionId and requestId as stable identifiers and safely handle duplicate final callbacks.
Reference strategy	Use a unique business reference per payment request where possible.
MSISDN format	Use international format beginning with country code, for example 2557XXXXXXX.
Clock synchronization	Keep merchant servers synchronized with a reliable time source to prevent timestamp validation failures.
Environment separation	Use different credentials and callback endpoints for UAT and production.

9. Support checklist before go-live

- Merchant profile approved and active.
- Required providerCode profiles configured and enabled.
- API client credentials issued and tested.
- Callback URL reachable over HTTPS and returns HTTP 200.
- Request signing implemented and verified.
- Status query tested for successful and failed scenarios.
- Business references and idempotency logic validated.